

SMART COLLEGE HUB

A Unified Digital Ecosystem for Academic Resource Sharing, Project Collaboration & Coding Skill Development

PROJECT TEAM

BEIT Final Year Students
Department of IT

SUPERVISOR

Faculty Advisor
Department of IT, NCIT

INSTITUTION

Nepal College of Info Tech (NCIT)
Affiliated with Pokhara University



Speaker Notes: Good morning respected external examiners, faculty members, and supervisor. We welcome you to our BEIT final project defense on "Smart College Hub", an integrated academic and collaboration web application developed at NCIT, Pokhara University.

Presentation Agenda

01 - 03

Foundations

Introduction, Problem Statement, and Core Objectives of Smart College Hub.

04 - 06

Architecture

System Comparative Analysis, Tech Stack Selection & System Architecture.

07 - 10

Core Modules

Auth RBAC, Study Materials, Peer Collaboration, and Coding Challenges.

11 - 13

Design & Testing

PostgreSQL ER Diagram, REST API Documentation (Swagger) & System Testing.

14 - 15

Demonstration

Visual walk-through of the operational user interface & core workflows.

16 - 18

Outcomes & Q&A

Future Scope, Conclusion, Learning Outcomes, and Q&A session.



Speaker Notes: Over the next 12 minutes, we will guide you through the problem background, system architecture, core engineering modules, database schema, testing results, and system demonstration before wrapping up with conclusions.

Project Overview & Context

- ✓ **Smart College Hub** is an all-in-one centralized digital platform engineered for higher education institutions in Nepal.
- ✓ Bridges the gap between **academic resource distribution**, peer-to-peer **project collaboration**, and **technical skill development**.
- ✓ Eliminates scattered social communication channels by establishing an official, structured campus community workspace.
- ✓ Designed with modern asynchronous API-first architecture, securing data integrity and high availability.

🎯 Campus Ecosystem Integration

Higher education demands seamless synergy between students, instructors, and practical coding practice.

 Notes & Syllabus

 Team Projects

 Coding Leaderboards



Speaker Notes: In current college environments, academic notes are scattered across Messaging groups, project teams form informally without institutional visibility, and coding practice happens on isolated platforms. Smart College Hub unifies these essential activities into one modern ecosystem.

⚠ Existing Challenges & Problem Statement



Fragmented Study Resources

Students face difficulty finding verified past question papers, notes, and syllabus PDFs due to unorganized messaging groups and broken link drives.



Inefficient Project Teaming

Students struggle to discover peers with complementary tech skills for final-year projects and hackathons, leading to uneven team skill distributions.



Lack of Peer Coding Practice

Absence of an internal, college-specific problem-solving environment to benchmark coding proficiency, prepare for technical interviews, and track peer ranks.



Speaker Notes: We identified three critical pain points through campus research: loss of valuable academic study materials between semesters, inefficient team discovery for engineering projects, and a lack of competitive coding practice tailored to our curriculum.

Project Objectives

⚙️ Core Technical Objectives

- ✓ Develop a RESTful API backend using [FastAPI](#) and [PostgreSQL](#) with robust role-based authorization (RBAC).
- ✓ Implement secure authentication using [JWT access tokens](#) and [Redis blocklist](#) for session invalidation.
- ✓ Build a responsive frontend application with [React 19](#), [TanStack Router](#), and [Tailwind CSS 4](#).

👥 Functional System Objectives

- ✓ Provide categorized material upload/search with subject filter tags and download tracking.
- ✓ Facilitate project team formation via skill matching and automated join request workflows.
- ✓ Host interactive algorithmic coding challenges with real-time test evaluations and college leaderboards.



Speaker Notes: Our objectives encompass both full-stack software engineering standards and specific academic functional outcomes. We aimed to deliver a production-grade system using modern frameworks while strictly adhering to security and data integrity best practices.

Comparative Analysis

Feature / Parameter	Legacy / Traditional Process	Smart College Hub (Proposed)
Resource Storage	Scattered Google Drive links & chat attachments	Centralized, verified cloud repository with metadata search
Project Collaboration	Informal personal contacts; no central registry	Role-based project listing with skill tags & request system
Coding Skill Assessment	Third-party platforms without college context	Integrated challenge engine with automated rank leaderboards
Access Control	None or basic shared passwords	OAuth2 / JWT RBAC (Student, Instructor, Admin)
Data Integrity & State	Unsynchronized local files	PostgreSQL relational database with Redis caching



Speaker Notes: This matrix highlights our technological advancements over traditional processes. By substituting informal tools with a dedicated web hub, we ensure permanent material accessibility, structured team recruiting, and automated coding evaluations.

Engineering Technology Stack

Backend & Infrastructure

 FastAPI (Python 3.11+)

 PostgreSQL & SQLAlchemy

 SQLAlchemy ORM

 Alembic DB Migrations

 Redis (Token Blocklist)

 PyJWT & Passlib (Bcrypt)

Frontend Ecosystem

 React 19 & TanStack Start

 TanStack Router (Type-Safe)

 Tailwind CSS v4

 TanStack Query (State)

 Zod & React Hook Form

 Recharts & Sonner Toast



Speaker Notes: We selected modern technologies: FastAPI on Python for high performance and automatic OpenAPI documentation, paired with React 19 and TanStack Start on the client for type-safe routing and state management.

High-Level System Architecture

-  **Client Layer:** Single Page Application rendered via React 19, consuming REST endpoints asynchronously.
-  **API Gateway / Auth Layer:** FastApi CORS middleware, OAuth2 bearer dependency, and Redis JWT token verification.
-  **Core Service Layer:** Modular backend handlers for Auth, Materials, Collaboration, and Coding Challenges.
-  **Data Layer:** Persistent PostgreSQL store managed via SQLAlchemy ORM with Redis caching.



 **Speaker Notes:** The architecture follows a clean 3-tier decoupling model. The frontend communicates over HTTPS via JSON REST APIs. Requests pass through custom FastAPI security middleware before interacting with business logic and database models.

Authentication & RBAC Module

JWT & Session Management

-  **Password Hashing:** Passlib with Bcrypt algorithm (salt factor 12) for secure credential storage.
-  **Stateless Tokens:** Issued OAuth2 JWT access tokens containing payload claims (User ID, Role, Expiry).
-  **Redis Blocklist:** Immediate token revocation upon logout by persisting JTI in in-memory Redis store.

Role-Based Access Control (RBAC)

-  **Student:** Access materials, submit challenge solutions, apply to project teams.
-  **Instructor:** Upload verified course materials, post official challenges, review projects.
-  **Admin:** System-wide user moderation, category creation, and analytics audit.

 **Speaker Notes:** Security is foundational. We implemented JWT authentication with a Redis-backed blocklist to address the classic stateless JWT logout challenge. Access control is strictly enforced via FastAPI dependency injection based on three distinct user roles.

Study Materials Module



Categorized Uploads

Upload syllabus PDFs, lab manuals, and past exam questions structured by Semester, Department, and Subject code.



Instant Index Search

Client-side Debounced search filtering across titles, tags, and faculty names powered by TanStack Query caching.



Audit & Download Tracking

Automated download counters track resource popularity and assist faculty in identifying key study demands.



Speaker Notes: The study materials module allows instructors and students to upload documents tagged by semester and subject. Built-in metadata search enables instant material discovery without page reloads.



Collaboration & Project Teams

-  **Project Workspace Creation:** Students can post project ideas specifying required roles (e.g., Frontend, Backend, ML Engineer).
-  **Join Request Workflow:** Applicants submit tailored join requests detailing their background experience.
-  **Member Management:** Project creators can accept/reject applications and assign team member permissions.
-  **Discussion Channel:** Built-in project activity logs and resource sharing links per project group.

Collaboration Lifecycle

1 Post Project & Skill Requirements

2 Peer Application & Skill Verification

3 Approval & Team Workspace Onboarding



Speaker Notes: The project collaboration module acts as an internal talent marketplace. Students publish project descriptions and needed skills, enabling interested peers to apply. Creators manage join requests seamlessly through a centralized dashboard.

</> Coding Challenges & Leaderboards



Problem Bank

Curated algorithmic challenges categorized by difficulty (Easy, Medium, Hard) and topic tags.



Solution Submission

Interactive code submission interface supporting multiple programming languages and test cases.



Live Leaderboard

Real-time ranking algorithm calculating aggregate user scores and problem completion speeds.



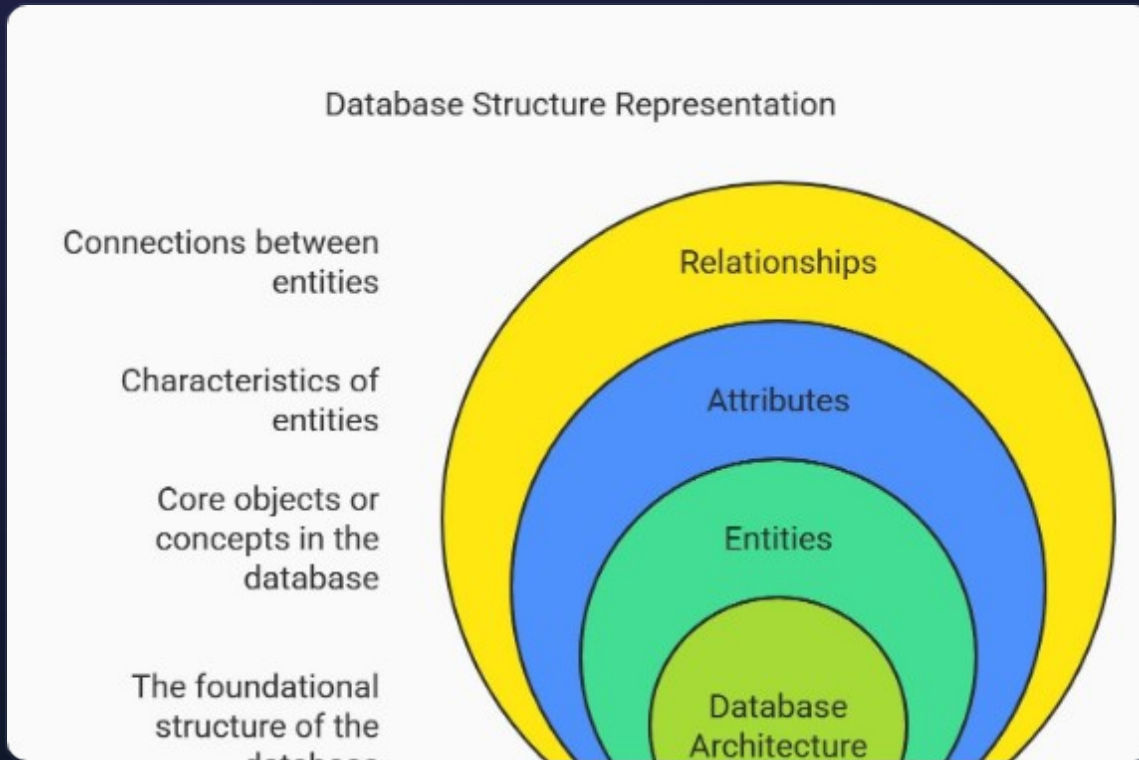
Progress Analytics

Personalized dashboards illustrating solved problem stats and accuracy metrics using Recharts.



Speaker Notes: To boost practical software skills, our coding challenge module hosts algorithmic problems. Submissions are validated against hidden test cases, automatically updating the campus leaderboard to foster healthy peer competition.

Database Architecture & ER Schema



Entity Relationships

- **User Entity:** Central table storing credentials, role enum, profile meta, linked to submissions and materials.
- **Material Entity:** Foreign key references to User (Uploader) and Category entities.
- **Project & Member Mapping:** 1-to-N relationship between Project and JoinRequests; N-to-M for Project Members.
- **Challenge & Submission:** 1-to-N mapping tracking score, runtime execution status, and user points.

 **Speaker Notes:** Our PostgreSQL schema designed via SQLModel maintains third normal form (3NF) compliance. It enforces foreign key constraints and index optimizations across primary tables: Users, Materials, Projects, Join Requests, Challenges, and Submissions.

API Specification & System Testing

OpenAPI / Swagger Documentation

- ✓ Auto-generated OpenAPI v3 endpoints via FastAPI at /docs.
- ✓ Strict Request/Response schema validation using Pydantic models.
- ✓ Interactive OAuth2 security authorization testing directly in UI.

Testing & Verification Metrics

- ✓ **Unit & Integration Testing:** Pytest suites verifying endpoints, DB models & auth helpers.
- ✓ **Validation Testing:** Zod schema checking for zero unexpected payload exceptions.
- ✓ **Performance:** Average API latency < 45ms with Redis caching enabled.

 **Speaker Notes:** All backend REST API routes were verified using automated Pytest suites and manual Swagger UI testing. Strong Pydantic request validation guarantees robust error handling and type-safe data serialization.

System Interface Demonstration

Authentication Screen

Clean JWT login interface with password visibility toggle and form validation.

Student Dashboard

Overview displaying recent study material downloads, active projects, and rank stats.

Resource Repository

Filterable table showing subject PDFs, upload dates, and direct download buttons.

Project Hub

Card view of team openings, required tech stack badges, and application forms.

Coding Arena

Interactive code editor layout with problem statement and test case results.

Global Leaderboard

Real-time student leaderboard displaying score metrics and badges.

 **Speaker Notes:** Here is an overview of our application screens. Designed with Tailwind CSS 4 and Radix UI primitives, the application delivers an intuitive, accessible experience across desktop and tablet viewpoints.

Future Scope & Roadmap



Real-Time WebSockets Chat

Integrating instant messaging within collaboration project groups using FastAPI WebSockets.



AI Material Recommendations

Embedding vector search (pgvector) to automatically suggest relevant study notes based on user syllabus.



Native Mobile Application

Developing cross-platform mobile apps with React Native consuming existing REST API routes.



Speaker Notes: Looking forward, we plan to extend Smart College Hub with real-time WebSocket messaging for project teams, AI-driven study resource recommendations, and cross-platform mobile support.

🚩 Conclusion & Key Achievements

🏆 Project Achievements

- ✓ Successfully delivered a fully functional, secure web application solving core campus challenges.
- ✓ Built robust full-stack architecture adhering to industry REST standard design patterns.
- ✓ Verified application stability with zero critical vulnerabilities during testing.

🎓 Team Engineering Outcomes

- 🔑 Mastered modern tech stacks: FastAPI, SQLAlchemy, React 19, and TanStack Router.
- 🛡️ Deepened expertise in authentication security, JWT blocklists, and RBAC strategies.
- 👥 Gained practical agile development, Git workflow, and system design experience.



Speaker Notes: In summary, Smart College Hub successfully addresses academic resource fragmentation and project collaboration bottlenecks at NCIT. The project allowed our team to master production full-stack engineering standards.

THANK YOU!

We sincerely appreciate your time and valuable guidance.

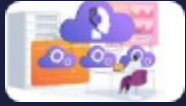
Department of Information Technology

Nepal College of Information Technology (NCIT)
Pokhara University, Nepal



Speaker Notes: Thank you distinguished members of the evaluation panel and faculty advisors. We are now open to your questions, feedback, and technical queries regarding Smart College Hub.

Image Sources



<https://media.istockphoto.com/id/2227037749/vector/cloud-native-microservices-abstract-concept-vector-illustration.jpg?s=612x612&w=0&k=20&c=qLTxfSApoS7ARSHykBjzddvvrQJYPet24vps0Gxc0EI=>

Source: www.istockphoto.com



https://ik.imagekit.io/upgrad1/abroad-images/imageCompo/images/__visualelection_3_AO03YB.jpeg

Source: www.upgrad.com